



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE QUIXADÁ
CURSO DE CIÊNCIA DA COMPUTAÇÃO
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

GUSTAVO GURGEL MEDEIROS

**GERAÇÃO DE MAPAS PARA JOGOS ROGUELIKE A PARTIR DE DESCRIÇÕES
TEXTUAIS UTILIZANDO MODELOS DE LINGUAGEM DE LARGA ESCALA**

QUIXADÁ

2025

GUSTAVO GURGEL MEDEIROS

GERAÇÃO DE MAPAS PARA JOGOS ROGUELIKE A PARTIR DE DESCRIÇÕES
TEXTUAIS UTILIZANDO MODELOS DE LINGUAGEM DE LARGA ESCALA

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Ciência da Computação
do Campus de Quixadá da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Ciência da Computação.

Orientador: Prof. Dr. Cristiano Bacelar
de Oliveira

QUIXADÁ

2025

RESUMO

O gênero de jogos *roguelike*, conhecido pela alta rejogabilidade proporcionada pela *Procedural Content Generation* (PCG), frequentemente apresenta desafios no que tange ao controle fino e à criação de conteúdo com intenções específicas por parte dos desenvolvedores. A manipulação de parâmetros complexos e pouco flexíveis em algoritmos de PCG tradicionais torna o processo de design pouco intuitivo. Este trabalho explora uma abordagem para superar essa limitação, propondo um sistema que utiliza *Large Language Model* (LLM) para gerar mapas para jogos *roguelike* a partir de descrições textuais em linguagem natural. A pesquisa visa desenvolver um *pipeline* que traduza a intenção do usuário em uma representação de mapa estruturada e funcional, empregando técnicas de engenharia de *prompt* para garantir a coerência dos níveis gerados, respeitando as convenções do gênero. Foi desenvolvido um *Vector Database* contendo os *tiles* (elementos gráficos) do jogo, componente essencial para a técnica de *Retrieval-Augmented Generation* (RAG). Para validar esta etapa, foi implementada uma *Graphical User Interface* (GUI) que permite ao usuário inserir uma consulta textual e visualizar os quatro *tiles* mais semanticamente relevantes, demonstrando a viabilidade de associar linguagem natural a ativos de jogo concretos. Este resultado inicial estabelece uma base técnica crucial para o sistema completo de geração de mapas "*text-to-map*".

Palavras-chave: Geração Procedural de Conteúdo. Modelos de Linguagem de Larga Escala. Jogos Roguelike. Geração de Mapas.

ABSTRACT

The *roguelike* game genre, known for the high replayability provided by PCG, often presents challenges regarding fine-grained control and the creation of content with specific intentions by developers. The manipulation of complex and inflexible parameters in traditional PCG algorithms makes the design process unintuitive. This work explores an approach to overcome this limitation by proposing a system that utilizes LLM to generate maps for *roguelike* games from textual descriptions in natural language. The research aims to develop a *pipeline* that translates user intent into a structured and functional map representation, employing *prompt* engineering techniques to ensure the coherence of the generated levels, respecting the genre's conventions. A *Vector Database* containing the game's *tiles* (graphical elements) was developed, an essential component for the RAG technique. To validate this stage, a GUI was implemented that allows the user to input a textual query and visualize the four most semantically relevant *tiles*, demonstrating the feasibility of associating natural language with concrete game assets. This initial result establishes a crucial technical foundation for the complete "*text-to-map*" map generation system.

Keywords: Procedural Content Generation. Large Language Models. Roguelike Games. Map Generation.

LISTA DE ABREVIATURAS E SIGLAS

ANN	<i>Artificial Neural Networks</i>
API	<i>Application Programming Interface</i>
ASCII	<i>American Standard Code for Information Interchange</i>
BERT	<i>Bidirectional Encoder Representations from Transformers)</i>
CNN	<i>Convolutional Neural Networks</i>
CSV	<i>Comma Separated Values</i>
GPT	<i>Generative Pre-trained Transformer</i>
GPU	<i>Graphics Processing Unit</i>
GUI	<i>Graphical User Interface</i>
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i>
LSTM	<i>Long Short-Term Memory</i>
NLP	<i>Natural Language Processing</i>
NPC	<i>Non-Player Character</i>
PCG	<i>Procedural Content Generation</i>
RAG	<i>Retrieval-Augmented Generation</i>
RNN	<i>Recurrent Neural Network</i>
RPG	<i>Role-Playing Game</i>
TPU	<i>Tensor Processing Unit</i>

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Objetivo Geral	7
1.2	Objetivos Específicos	7
2	FUNDAMENTAÇÃO TEÓRICA	8
2.1	Roguelikes e Roguelites	8
2.2	PCG em Jogos	11
2.2.1	<i>Definição e Objetivos da PCG</i>	11
2.2.2	<i>Aplicações da PCG em Jogos</i>	12
2.2.3	<i>Vantagens e Desafios da PCG</i>	12
2.2.4	<i>Técnicas Comuns de PCG</i>	14
2.3	LLMs	16
2.3.1	<i>Definição e Evolução Histórica dos LLMs</i>	16
2.3.2	<i>Transformers e Mecanismo de Atenção</i>	17
2.3.3	<i>Treinamento e Capacidades dos LLMs</i>	17
2.3.4	<i>LLMs Aplicados à PCG</i>	18
3	TRABALHOS RELACIONADOS	20
4	METODOLOGIA	22
4.1	Visão Geral da Metodologia	22
4.2	Seleção de Técnicas de Engenharia de Prompt	22
4.3	Modelagem do Pipeline de Geração	23
4.4	Definição das Estruturas de Dados de Entrada e Saída	25
4.5	Seleção de Ferramentas e Tecnologias	26
4.6	Implementação do Gerador de Mapas	26
4.7	Implementação do Visualizador de Mapas	26
4.8	Definição das Métricas de Validação	27
4.9	Validação e Análise dos Resultados	27
5	RESULTADOS PRELIMINARES	28
5.1	Implementação da Primeira Versão do Banco de Tiles	28
6	PLANO DE TRABALHO	31
	REFERÊNCIAS	32

1 INTRODUÇÃO

Os jogos digitais representam uma **área de significativo fascínio** na sociedade contemporânea, com aplicações que abrangem desde o entretenimento até fins educacionais. A indústria de jogos eletrônicos tem demonstrado um crescimento notável, consolidando-se como um mercado de alcance global sem precedentes (WIJMAN, 2022).

Nesse contexto, **diversos jogos do gênero roguelike** têm alcançado grande sucesso, destacando-se pela capacidade de proporcionar experiências desafiadoras e imersivas. O gênero *roguelike*, um subgênero de *Role-Playing Game* (RPG), caracteriza-se fundamentalmente pela exploração de **masmorras geradas aleatoriamente**, combate tipicamente baseado em turnos e, **notavelmente**, a morte permanente do personagem, que impõe o recomeço do jogo após uma derrota (LAURINDO *et al.*, 2024).

Um elemento fundamental que define a experiência em muitos jogos *roguelike* é a PCG. PCG é uma técnica que utiliza algoritmos para criar automaticamente conteúdos de jogo, como mapas e cenários, sendo fundamental para garantir que cada sessão de jogo seja única e ofereça alta rejogabilidade (SEPÄNMAA, 2024). Embora a PCG seja poderosa para gerar uma vasta quantidade de conteúdo, um de seus desafios históricos reside na dificuldade de controlar o resultado. Tradicionalmente, os desenvolvedores manipulam parâmetros complexos e abstratos para guiar o processo gerador, o que muitas vezes torna a criação de conteúdo com intenções específicas uma tarefa pouco intuitiva e iterativa (LAZARIDIS; FRAGULIS, 2024).

A busca por maior **controlabilidade** na PCG tem sido um campo ativo de pesquisa. Contudo, gerar conteúdo que reflita intenções específicas e restrições de alto nível permanece um desafio. Recentemente, os avanços em LLMs abriram novas e promissoras formas para abordar este problema. A capacidade inerente dos LLMs de compreender e processar linguagem natural oferece uma interface fundamentalmente nova para a geração de conteúdo: em vez de ajustar parâmetros, os desenvolvedores ou mesmo os jogadores poderiam simplesmente descrever o conteúdo desejado textualmente.

Essa abordagem, frequentemente denominada "*text-to-level*" ou "*text-to-map*", transforma o LLM em um tradutor de intenções humanas para estruturas de jogo concretas. Um trabalho pioneiro nesta área, MarioGPT, demonstrou a viabilidade de treinar um LLM para gerar níveis de Super Mario Bros a partir de descrições simples como "muitos canos, poucos inimigos, elevação baixa" (SUDHAKARAN *et al.*, 2023). Este estudo provou que os LLMs podem não apenas gerar sequências textuais, mas também produzir conteúdo estruturado e funcional para

jogos, abordando um dos principais desafios da PCG.

Inspirado por esses avanços, este trabalho busca investigar a aplicação dessa abordagem no domínio dos jogos *roguelike*. Diferentemente dos jogos de plataforma, os *roguelikes* possuem restrições estruturais distintas, como a conectividade entre salas, a disposição de corredores e a distribuição lógica de inimigos e tesouros. O objetivo é, portanto, explorar como os LLMs podem ser instruídos, via descrições textuais, a gerar mapas coerentes e jogáveis que respeitem as convenções do gênero.

Este estudo busca desenvolver um sistema que utilize um LLM para interpretar descrições textuais e produzir uma representação de mapa estruturada (por exemplo, em formato *JavaScript Object Notation* (JSON)), que possa ser posteriormente renderizada por um motor de jogo. A pesquisa se concentrará na eficácia de técnicas de engenharia de *prompt* para guiar o modelo, garantindo que os mapas gerados não apenas correspondam à descrição fornecida, mas também sejam funcionalmente válidos dentro das regras de um jogo *roguelike*. Acredita-se que tal abordagem tem o potencial de oferecer um método mais intuitivo e expressivo para a criação de conteúdo procedural, contribuindo com conhecimento técnico e metodológico.

1.1 Objetivo Geral

Desenvolver um sistema que recebe descrições textuais e, utilizando um LLM, traduz em um nível estruturado e jogável de um jogo no estilo *roguelike*.

1.2 Objetivos Específicos

Para alcançar o objetivo geral proposto, os seguintes objetivos específicos foram delineados:

- Investigar e aplicar técnicas de engenharia de *prompt* para guiar o LLM de forma eficaz, assegurando que os mapas gerados sejam coerentes com a descrição textual e respeitem as restrições e convenções do gênero *roguelike*.
- Desenvolver um *pipeline* de geração que receba uma descrição textual como entrada e utilize um LLM para produzir uma representação de mapa estruturada, contendo elementos como salas, corredores, inimigos e itens.
- Implementar um protótipo funcional que integre o *pipeline* de geração, capaz de processar os dados do mapa e renderizá-los visualmente, servindo como prova de conceito da abordagem "*text-to-map*".

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, serão apresentados os principais conceitos fundamentais para o desenvolvimento deste trabalho. A Seção 2.1 trata dos aspectos históricos e mecânicas fundamentais dos gêneros *roguelike* e *roguelite*. Na Seção 2.2, discute-se a PCG, abordando sua definição, objetivos, aplicações, vantagens, desafios e técnicas comuns. Por fim, a Seção 2.3 oferece uma descrição sobre os LLMs, considerando sua definição, evolução, arquitetura, treinamento, capacidades e aplicação na PCG.

2.1 Roguelikes e Roguelites

O gênero *roguelike* compreende uma categoria de jogos de RPG eletrônicos caracterizados por uma série de mecânicas distintas, das quais a *permadeath* e a geração procedural de níveis são as mais proeminentes. O termo deriva diretamente do jogo seminal *Rogue*^{1,2}, desenvolvido por Michael Toy, Glenn Wichman e Ken Arnold e lançado em 1980. *Rogue* surgiu em um contexto onde jogos de aventura baseados em texto, como *Colossal Cave Adventure*, e RPGs de mesa, como *Dungeons & Dragons*, eram populares, mas a representação gráfica e a interatividade dinâmica em computadores pessoais ainda estavam em seus estágios iniciais (HARRIS, 2020).

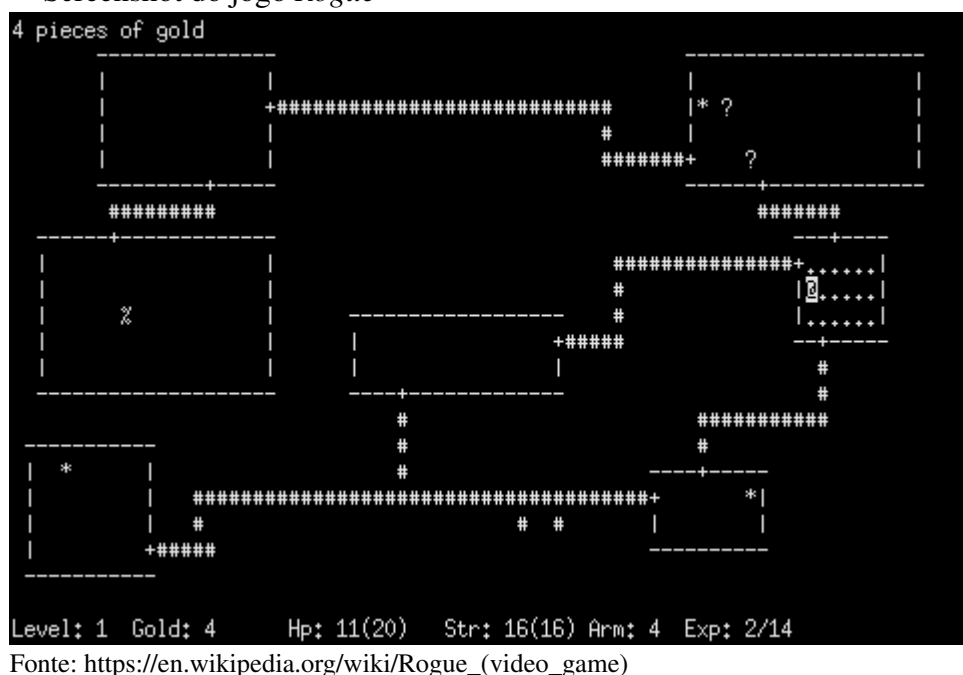
Em *Rogue*, o jogador assume o papel de um aventureiro explorando uma masmorra perigosa com o objetivo de recuperar o "Amuleto de Yendor" e retornar à superfície. O jogo utilizava caracteres no formato *American Standard Code for Information Interchange* (ASCII) para representar graficamente o personagem do jogador (geralmente um '@'), monstros (representados por letras diversas), itens e o layout da masmorra, como ilustrado na Figura 1. Esta abordagem visual, embora primitiva para os padrões atuais, era inovadora por permitir uma representação espacial do ambiente do jogo, que era gerado aleatoriamente a cada nova partida, uma característica fundamental que definiria o gênero. As influências de *Dungeons & Dragons* e *Colossal Cave Adventure* são perceptíveis na temática de exploração de masmorras e coleta de tesouros.

O impacto de *Rogue* foi significativo, estabelecendo as fundações para um gênero que persistiria e evoluiria por décadas. Desenvolvido para terminais *Unix* e utilizando a biblioteca

¹ <https://www.roguebasin.com/index.php/Rogue>

² <https://britzl.github.io/roguearchive/files/misc/EpyxRogueDOSManual/manual.htm>

Figura 1 – Screenshot do jogo *Rogue*



Curses^{3,4} para manipulação de tela, *Rogue* introduziu avanços importantes não apenas para os RPGs de computador, mas para o gênero de jogos de PC em geral. A geração procedural, a trama básica de exploração e retorno, e a exibição baseada em texto tornaram-se características centrais e recorrentes nos jogos que se seguiram e que se autodenominariam *roguelikes* (HARRIS, 2020).

Um dos pilares fundamentais do gênero *roguelike* é a mecânica de *permadeath*. Essa característica implica que, quando o personagem do jogador morre, todo o progresso naquela partida específica é perdido, e o jogador deve recomeçar do zero. A *permadeath* intensifica a tensão e o valor de cada decisão tomada pelo jogador, pois um erro pode ter consequências fatais e irreversíveis para aquela sessão de jogo. Este ciclo de tentativa, falha e aprendizado é central para a experiência *roguelike*, forçando os jogadores a internalizar as mecânicas do jogo e a desenvolverem suas próprias habilidades e estratégias, em vez de dependerem apenas da progressão do personagem.

A natureza punitiva da *permadeath* é frequentemente vista como um elemento que torna as escolhas e os riscos mais significativos. Em muitos jogos de RPG tradicionais, a morte do personagem é uma inconveniência temporária, resolvida com o carregamento de um jogo salvo. Nos *roguelikes* clássicos, no entanto, a perda do personagem significa o fim daquela jornada específica, e o jogador leva consigo apenas o conhecimento adquirido para a próxima tentativa. Este foco na habilidade do jogador, em detrimento da acumulação de recursos ou níveis

³ <https://www.ibm.com/docs/en/aix/7.2.0?topic=concepts-curses-library>

⁴ https://en.wikipedia.org/wiki/Category:Free_software_that_uses_ncurses

do personagem que persistem entre as mortes, é um diferencial marcante do gênero (BOTTA, 2011).

De forma complementar à *permadeath*, a PCG é o outro pilar essencial dos *roguelikes*. Em *Rogue*, e em seus sucessores, os níveis da masmorra, a localização de inimigos, itens e tesouros são gerados algoritmicamente a cada nova partida. Isso garante uma alta rejogabilidade, pois cada sessão de jogo apresenta um desafio único e imprevisível. Mesmo os desenvolvedores de *Rogue* não poderiam prever a configuração exata de uma masmorra, o que contribuía para a sensação de exploração genuína. A PCG em *roguelikes* não se limita apenas a reorganizar elementos pré-existentes, mas pode criar combinações emergentes de desafios e oportunidades que testam a adaptabilidade do jogador (SHORT; ADAMS, 2017).

Por muitas décadas, o gênero *roguelike* permaneceu um nicho, apreciado por uma comunidade dedicada de fãs, mas foi nesta última década que ele emergiu para o mainstream, influenciando uma vasta gama de jogos independentes e até mesmo títulos de grande orçamento. Com essa popularização, surgiu o termo *roguelite*⁵ para descrever jogos que incorporam elementos chave dos *roguelikes*, como a *permadeath* e a geração procedural, mas que se desviam da fórmula clássica, muitas vezes introduzindo sistemas de meta-progressão⁶. Essa meta-progressão permite que certos desbloqueios, habilidades ou recursos sejam carregados entre as partidas, suavizando a curva de dificuldade e oferecendo uma sensação de progresso contínuo mesmo após a falha.

Figura 2 – Dead Cells (2018)



Fonte: <https://dead-cells.com/>

⁵ <https://www.gamedeveloper.com/design/the-roguelike-debate—roguelikes-vs-roguelites>

⁶ <https://hackernoon.com/what-are-roguelites-a-deep-dive>

Os *roguelites* focam frequentemente em alcançar um final definido, em contraste com os *roguelikes* clássicos, onde o foco está mais na jornada e na superação de desafios imediatos de cada partida. A persistência de elementos entre as tentativas, como melhorias de atributos base, desbloqueio de novas armas ou habilidades, mecânicas centrais em títulos como o visualmente impactante *Dead Cells* (Figura 2), torna os *roguelites* mais acessíveis a um público mais amplo, que pode se sentir frustrado pela natureza implacável da *permadeath* total. Outros títulos como *Spelunky*⁷, *FTL: Faster Than Light*⁸, e *The Binding of Isaac*⁹ são também exemplos proeminentes que aplicaram a fórmula *roguelike* a diferentes gêneros, introduzindo variações que levaram à necessidade de uma nova categorização, embora o debate sobre as definições exatas de *roguelike* e *roguelite* continue sendo debatido entre fãs e desenvolvedores.

2.2 PCG em Jogos

A PCG tem se estabelecido como uma área de crescente importância e aplicação no desenvolvimento de jogos digitais. Esta seção visa apresentar uma visão geral sobre a PCG, abordando sua definição, objetivos, aplicações, vantagens, desafios e as técnicas mais comuns empregadas, com ênfase em sua relevância para a geração de níveis e narrativas, especialmente no contexto de jogos do gênero *roguelike*.

2.2.1 Definição e Objetivos da PCG

A PCG refere-se fundamentalmente à criação de conteúdo de jogo de forma algorítmica, em vez de manualmente. Segundo Togelius *et al.* (2011a), PCG é a geração de conteúdo de jogo por algoritmos, abrangendo desde elementos visuais até mecânicas de jogo. Smith *et al.* (2011) complementa esta visão, destacando que o objetivo da PCG não é apenas automatizar a criação, mas também possibilitar a emergência de novas formas de experiência de jogo, como a rejogabilidade infinita ou mundos de jogo vastos e únicos. O objetivo principal da PCG é, portanto, utilizar processos computacionais para produzir uma grande quantidade e variedade de conteúdo, muitas vezes com recursos de desenvolvimento limitados, ou para criar experiências que seriam impossíveis ou impraticáveis de serem desenvolvidas por métodos tradicionais de design manual. Para o escopo deste trabalho, a PCG é entendida como o conjunto de técnicas e

⁷ <https://www.spelunkyworld.com/>

⁸ <https://subsetgames.com/ftl.html>

⁹ https://store.steampowered.com/app/113200/The_Binding_of_Isaac/

algoritmos empregados para gerar, automaticamente ou com intervenção mínima do designer, elementos como níveis, narrativas e outros componentes de jogos.

2.2.2 Aplicações da PCG em Jogos

A PCG encontra um vasto campo de aplicação no desenvolvimento de jogos, permeando diversos aspectos da experiência do jogador. Dentre os tipos de conteúdo mais comumente gerados proceduralmente, destacam-se os níveis e ambientes, que constituem um foco central deste trabalho, devido à sua importância para a exploração e desafio em jogos (FREIKNECHT, 2020). Além da arquitetura de níveis, a PCG é amplamente utilizada para criar itens com atributos variados, posicionar inimigos e gerar *Non-Player Character* (NPC) com características distintas. A geração também se estende a elementos audiovisuais, como texturas, modelos 3D, música e efeitos sonoros, contribuindo para a riqueza sensorial dos mundos virtuais. Outra aplicação relevante é a criação de missões e *quests*, que podem ser adaptadas dinamicamente ao progresso do jogador ou a eventos do jogo.

Diversos jogos comerciais e independentes utilizam PCG de forma proeminente para alcançar seus objetivos de design. Títulos como *Spelunky* e seus sucessores são exemplos clássicos de como a PCG pode criar níveis desafiadores e altamente rejogáveis. *No Man's Sky*¹⁰ utiliza PCG para gerar um universo de planetas, cada um com sua flora, fauna e topografia únicas, oferecendo uma escala de exploração virtualmente infinita. *Minecraft*¹¹ é outro exemplo icônico, onde vastos mundos são gerados proceduralmente, permitindo que os jogadores explorem e modifiquem o terreno de maneiras criativas. O gênero *roguelike*, desde seus precursores como *Rogue* até títulos modernos, é intrinsecamente ligado à PCG, utilizando-a para gerar masmorras, itens e encontros, garantindo que cada partida seja uma nova experiência (FREIKNECHT, 2020).

2.2.3 Vantagens e Desafios da PCG

A adoção da PCG no desenvolvimento de jogos traz consigo um conjunto de vantagens significativas, mas também impõe desafios consideráveis que precisam ser gerenciados pelos desenvolvedores. Entre as principais vantagens, figura a capacidade de oferecer alta rejogabilidade, uma vez que o conteúdo pode variar a cada nova sessão de jogo (LAZARIDIS; FRAGULIS, 2024). A PCG permite a criação de mundos vastos e variados que seriam inviáveis de serem

¹⁰ <https://www.nomanssky.com/>

¹¹ <https://www.minecraft.net/>

Figura 3 – Mundos do *Minecraft* como este são todos gerados algoritmicamente.



Fonte: https://minecraft.wiki/w/World_generation

construídos manualmente, como observado em *No Man's Sky*. Outro benefício importante é a eficiência de armazenamento, já que o conteúdo é gerado em tempo de execução ou durante o desenvolvimento a partir de algoritmos compactos, em vez de armazenar grandes volumes de dados pré-fabricados (FREIKNECHT, 2020). Adicionalmente, a PCG possui o potencial para adaptação dinâmica do conteúdo às ações ou ao perfil do jogador, criando experiências mais personalizadas.

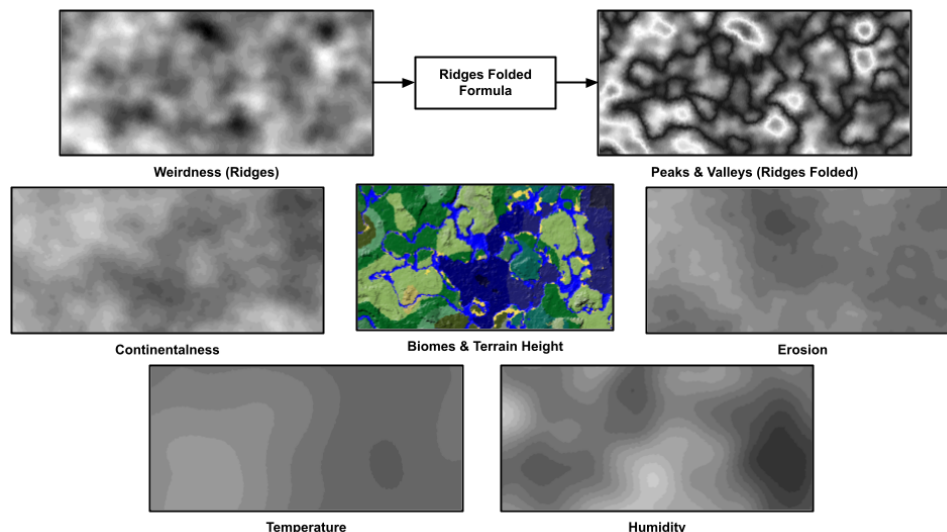
Contudo, a PCG não está isenta de desafios. O controle de qualidade do conteúdo gerado pode ser complexo, pois é preciso garantir que os resultados sejam consistentemente bons, esteticamente agradáveis e livres de artefatos indesejados ou bugs (LAZARIDIS; FRAGULIS, 2024). A garantia de jogabilidade, incluindo a solvabilidade de níveis ou quebra-cabeças e a "justiça" das situações de jogo, é outra preocupação central; um nível gerado proceduralmente deve ser sempre completável e oferecer um desafio adequado (FREIKNECHT, 2020). A previsibilidade excessiva pode minar a sensação de descoberta, enquanto a aleatoriedade descontrolada pode gerar conteúdo incoerente ou frustrante. Finalmente, um dos desafios mais debatidos é a dificuldade em replicar o "toque humano", ou seja, criar conteúdo com significado profundo, intenção autoral forte ou nuances emocionais complexas que são frequentemente associadas ao design manual (LAZARIDIS; FRAGULIS, 2024).

2.2.4 Técnicas Comuns de PCG

O campo da PCG abrange uma diversidade de técnicas algorítmicas, cada uma com suas particularidades e aplicabilidades. Para a geração de níveis, foco deste trabalho, e também para narrativas simples, algumas abordagens se destacam antes da introdução de LLMs, que serão discutidos posteriormente. É importante ressaltar que muitas dessas técnicas podem ser combinadas para alcançar resultados mais complexos e controláveis.

Algoritmos baseados em ruído, como *Perlin Noise* (PERLIN, 1985) e *Simplex Noise* (PERLIN, 2001), são frequentemente utilizados para gerar terrenos, texturas, mapas de altura e outras distribuições espaciais que requerem uma aparência orgânica e natural. Esses algoritmos produzem padrões pseudoaleatórios que podem ser parametrizados para controlar características como frequência e amplitude, resultando em formações geográficas verossímeis ou distribuições de recursos. Um exemplo prático disso pode ser observado na Figura 4, que ilustra as funções de densidade (com *3D noise*) utilizadas na geração de biomas e altura de terreno no jogo *Minecraft*.

Figura 4 – Exemplo de funções de densidade que utilizam ruído 3D para geração de biomas e altura de terreno no *Minecraft*

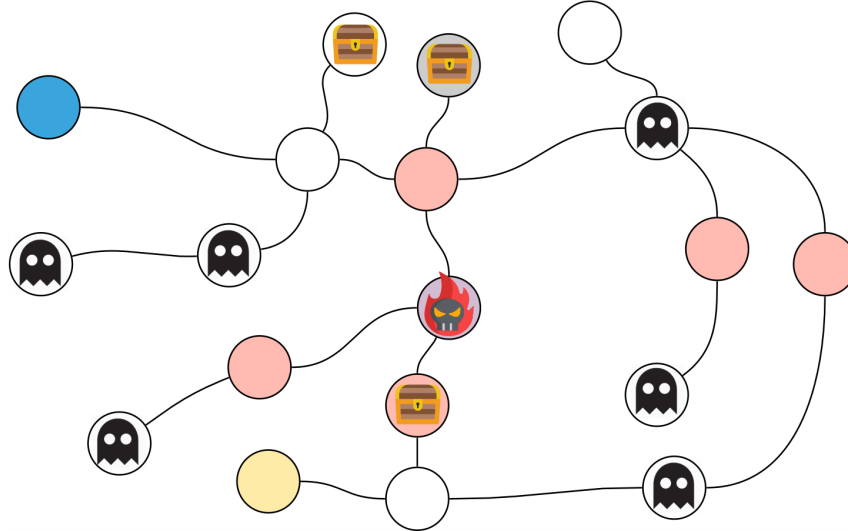


Fonte: https://minecraft.wiki/w/World_generation

Gramáticas formais, incluindo *L-Systems* (LINDENMAYER, 1968), *Shape Grammars* (STINY; GIPS, 1971) e *Graph Grammars* (EHRIG *et al.*, 1973), oferecem um meio poderoso para definir regras de construção que podem gerar estruturas complexas e recursivas. L-Systems são notáveis para modelar o crescimento de plantas e fractais. Shape Grammars operam sobre formas geométricas, permitindo a criação de estilos arquitetônicos ou layouts. Graph Grammars, particularmente relevantes para a geração de níveis em jogos do tipo *roguelike*,

manipulam grafos para definir a estrutura e conectividade de salas, corredores e áreas em um mapa, como pode ser observado na Figura 5.

Figura 5 – Exemplo de mapa gerado utilizando gramáticas de grafos



Fonte: (SOUZA *et al.*, 2019)

Técnicas baseadas em agentes e simulação permitem que entidades autônomas "esculpam" o ambiente ou distribuam elementos de forma emergente. Um exemplo clássico é o algoritmo *Random Walk* (PEARSON, 1905), onde um agente se move aleatoriamente por uma grade, "cavando" túneis ou caminhos, frequentemente usado na geração de cavernas ou masmorras simples (KOESNAEDI; ISTIONO, 2022). Simulações mais complexas podem envolver múltiplos agentes com comportamentos distintos para criar ecossistemas ou padrões urbanos.

A *PCG Search-Based* (TOGELIUS *et al.*, 2011b) utiliza algoritmos de otimização, como algoritmos genéticos, evolução diferencial ou *simulated annealing*, para explorar o vasto espaço de possíveis conteúdos e encontrar aqueles que melhor satisfazem um conjunto de critérios de qualidade ou restrições de design, definidos por uma função de avaliação. Esta abordagem é útil quando o designer tem objetivos claros sobre as propriedades desejadas do conteúdo, mas a geração direta é complexa.

Finalmente, abordagens baseadas em Aprendizado de Máquina, anteriores à proeminência dos LLMs, já exploravam a capacidade de aprender padrões a partir de dados de exemplo para gerar novo conteúdo. Isso inclui técnicas como *Artificial Neural Networks* (ANN) (MCCULLOCH; PITTS, 1943), Árvore de Decisão (LOH, 2011), ou Cadeias de Markov (LEVIN; PERES, 2017) para gerar sequências, como layouts de níveis, trechos de música ou

comportamento de inimigos (WERNECK; CLUA, 2020). A ascensão dos LLMs pode ser vista como uma evolução natural e mais poderosa desta vertente, especialmente para conteúdo com estrutura sequencial ou semântica complexa, como narrativas.

2.3 LLMs

Os LLMs representam uma classe de modelos de inteligência artificial que demonstraram capacidades notáveis no processamento e geração de linguagem natural. Sua ascensão tem sido impulsionada por avanços em arquiteturas de redes neurais, disponibilidade de grandes volumes de dados textuais e poder computacional crescente.

2.3.1 Definição e Evolução Histórica dos LLMs

Os LLMs são modelos de inteligência artificial treinados em vastas quantidades de dados textuais, o que lhes confere a capacidade de entender, gerar e manipular a linguagem humana de forma coerente e contextualmente relevante (HADI *et al.*, 2023). Eles são projetados para prever a próxima palavra ou caractere em uma dada sequência de texto, uma tarefa fundamental no *Natural Language Processing* (NLP).

A evolução dos LLMs pode ser rastreada desde os primeiros modelos de linguagem estatísticos, como n-grams (BROWN *et al.*, 1992), que se baseavam em contagens de frequência de sequências de palavras. Posteriormente, as *Recurrent Neural Network* (RNN) e suas variantes, como *Long Short-Term Memory* (LSTM) (HOCHREITER; SCHMIDHUBER, 1997), trouxeram melhorias ao capturar dependências de longo alcance no texto, embora ainda enfrentassem desafios com sequências muito longas e paralelização do treinamento (HADI *et al.*, 2023).

Um marco fundamental nessa evolução foi a introdução da arquitetura Transformer no artigo "Attention is All You Need" por Vaswani *et al.* (2017). Essa arquitetura, baseada unicamente em mecanismos de atenção, dispensou a recorrência e permitiu um treinamento mais eficiente e paralelizável em grandes volumes de dados. A partir dos Transformers, surgiram modelos influentes como *Bidirectional Encoder Representations from Transformers* (BERT) (DEVLIN *et al.*, 2019) e a série *Generative Pre-trained Transformer* (GPT) (RADFORD *et al.*, 2018), que estabeleceram novos paradigmas no NLP e impulsionaram o desenvolvimento dos LLMs modernos (HADI *et al.*, 2023).

2.3.2 Transformers e Mecanismo de Atenção

A arquitetura Transformer, conforme introduzida por Vaswani *et al.* (2017), é a base da maioria dos LLMs contemporâneos. Diferentemente das arquiteturas baseadas em RNN ou *Convolutional Neural Networks* (CNN) (LECUN *et al.*, 2002), os Transformers processam todos os tokens¹² de uma sequência de entrada simultaneamente, o que facilita a paralelização e o treinamento em hardware especializado como *Graphics Processing Unit* (GPU) e *Tensor Processing Unit* (TPU) (LIN *et al.*, 2022).

Um dos seus componentes mais revolucionários é o mecanismo de *self-attention*. Este mecanismo permite que o modelo, ao processar uma *token* em uma sequência, pondere a importância de todos os outros *tokens* na mesma sequência. Essencialmente, ele calcula "scores de atenção" que determinam o quanto cada palavra deve contribuir para a representação da palavra atual. Isso possibilita que o modelo capture dependências de longo alcance e entenda o contexto de forma mais eficaz, mesmo em sequências de texto extensas (VASWANI *et al.*, 2017; NAVEED *et al.*, 2023). A capacidade de processar sequências em paralelo, combinada com o poder do mecanismo de auto-atenção para modelar relações contextuais complexas, tornou os Transformers particularmente adequados para o treinamento em grandes volumes de dados textuais, resultando nas capacidades impressionantes dos LLMs atuais (LIN *et al.*, 2022).

2.3.3 Treinamento e Capacidades dos LLMs

O treinamento de LLMs geralmente envolve duas fases principais. A primeira é o pré-treinamento, onde o modelo é exposto a enormes quantidades de dados textuais não rotulados (tais como, livros, artigos, websites). Durante essa fase, o modelo aprende a prever palavras mascaradas em uma sentença (como no BERT) ou a próxima palavra em uma sequência (como no GPT), o que o força a aprender padrões linguísticos, conhecimento de mundo e relações semânticas complexas (LIU *et al.*, 2024; NAVEED *et al.*, 2023).

Após o pré-treinamento, os modelos podem passar por um processo de ajuste *fine-tuning* para tarefas específicas. Nesta etapa, o modelo pré-treinado é treinado adicionalmente em um dataset menor e específico da tarefa (por exemplo, tradução, sumarização, resposta a perguntas), com dados rotulados. Isso adapta o conhecimento geral adquirido durante o pré-treinamento para a tarefa desejada (LIU *et al.*, 2024).

¹² Um token pode ser uma palavra, uma subpalavra (um subconjunto de uma palavra) ou até mesmo um único caractere. Fonte: <https://developers.google.com/machine-learning/crash-course/llm/transformers>

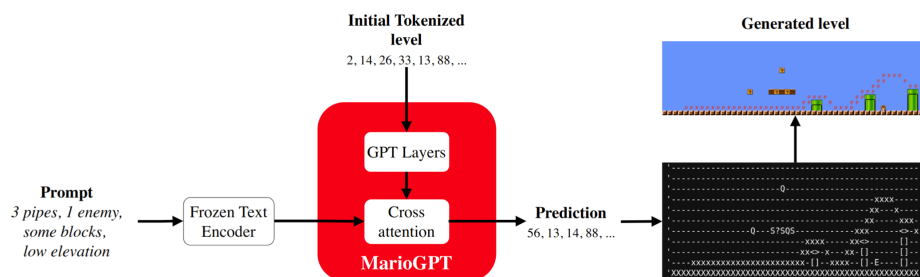
Como resultado, os LLMs exibem uma vasta gama de capacidades, incluindo a geração de texto coerente e criativo, tradução entre idiomas, sumarização de documentos longos, resposta a perguntas de forma informativa, e até mesmo a escrita de código (HADI *et al.*, 2023). Uma característica notável é a emergência de capacidades complexas, como o raciocínio em múltiplos passos e o aprendizado em poucos exemplos (*few-shot learning*) ou mesmo sem exemplos prévios (*zero-shot learning*), onde o modelo pode realizar tarefas para as quais não foi explicitamente treinado, apenas com base em uma descrição da tarefa e, opcionalmente, alguns exemplos fornecidos no prompt (VASWANI *et al.*, 2017; NAVEED *et al.*, 2023).

2.3.4 LLMs Aplicados à PCG

As capacidades dos LLMs, especialmente sua proficiência na compreensão e geração de linguagem natural, alinham-se de forma promissora com as necessidades da Geração Procedural de Conteúdo (PCG) em jogos (MALEKI; ZHAO, 2024). A PCG visa criar conteúdo de jogo (níveis, narrativas, itens, etc.) algoritmicamente, e os LLMs oferecem novas avenidas para enriquecer e personalizar esse processo.

Um dos potenciais mais diretos dos LLMs na PCG é a geração de elementos textuais. Isso inclui a criação de narrativas dinâmicas que se adaptam às ações do jogador, diálogos interativos e realistas para NPCs, descrições ricas e detalhadas de itens, locais e personagens, e histórias de fundo para o mundo do jogo (ROLIM, 2023; KOPPEN, 2024). A capacidade dos LLMs de manter a coerência e o contexto ao longo de textos extensos é particularmente valiosa para esses fins.

Figura 6 – Arquitetura do sistema MarioGPT. Um prompt textual é processado por um encoder de texto, cujas saídas são usadas em um mecanismo de atenção cruzada com as camadas GPT. As camadas GPT também recebem um nível tokenizado inicial. O sistema então prediz a continuação do nível tokenizado, que é convertido para um nível de jogo visual.



Fonte: (SUDHAKARAN *et al.*, 2023)

Além de elementos puramente textuais, os LLMs também têm potencial para influenciar a geração de elementos estruturais, ainda que indiretamente. Por exemplo, um LLM pode gerar descrições textuais detalhadas de *layouts* de níveis, sequências de eventos de uma missão, ou regras de um sistema de jogo. Essas descrições podem então ser interpretadas por outros sistemas ou algoritmos de PCG para traduzi-las em conteúdo de jogo concreto (MALEKI; ZHAO, 2024). Pesquisas recentes, como o estudo MarioGPT (cuja arquitetura é ilustrada na Figura 6), exploraram o uso de LLMs para gerar níveis de jogos 2D a partir de *prompts* textuais, indicando um caminho para a personalização de conteúdo de jogo com base nas preferências do usuário expressas em linguagem natural (KOPPEN, 2024).

3 TRABALHOS RELACIONADOS

A utilização de Modelos de LLM na PCG em jogos é uma área de pesquisa em expansão. Diversos trabalhos têm explorado o potencial dos LLMs para criar não apenas elementos de jogo isolados, mas também mundos e narrativas coesas. Nesta seção, são apresentados trabalhos relevantes que abordam a geração de conteúdo de jogos com LLMs, estabelecendo um paralelo com a proposta deste TCC e contextualizando suas contribuições. A análise segue uma ordem decrescente de afinidade, partindo do trabalho mais diretamente relacionado até os que apresentam semelhanças em aspectos mais específicos da abordagem.

O trabalho de Nasir *et al.* (2024), intitulado *Word2World*, apresenta uma abordagem que se assemelha de forma significativa aos objetivos deste TCC. Os autores propõem um sistema que utiliza LLMs para gerar mundos de jogo 2D jogáveis a partir de uma história inicial. O processo é realizado em um *pipeline* multifásico que inclui a criação da história, a extração de informações cruciais (como personagens, *tiles*¹ e objetivos), a geração do mundo e um ciclo de *feedback* para avaliação e refinamento. A principal semelhança com o presente trabalho reside no objetivo central de conectar uma narrativa de alto nível à geração de um nível de jogo concreto e coerente, utilizando um LLM para mediar ambas as tarefas. A principal diferença, contudo, está no escopo de aplicação. Enquanto *Word2World* foca em jogos 2D de visão superior de forma genérica, este TCC se especializa no gênero *roguelike*, que possui restrições e características próprias, como a alta rejogabilidade e a estrutura de masmorras, aspectos que exigem uma adaptação e especialização do processo de geração.

De forma similar, o trabalho *MarioGPT* de Sudhakaran *et al.* (2023) demonstra a viabilidade da geração de níveis a partir de descrições textuais, focando no gênero de plataforma. O sistema utiliza um modelo GPT2 ajustado para criar fases do jogo Super Mario Bros². a partir de *prompts* simples, como "muitos canos, poucos inimigos, elevação baixa". A semelhança com este TCC reside na abordagem "*text-to-level*", provando que LLMs podem interpretar descrições e produzir conteúdo estruturado e funcional para um jogo. No entanto, a distinção fundamental está no gênero e na complexidade estrutural. Enquanto os níveis de plataforma são tipicamente lineares, os mapas de *roguelike* exigem uma estrutura de grafo com salas interconectadas, um desafio de conectividade e *layout* que não é o foco principal de *MarioGPT*.

A dissertação de Auxtero (2023) explora a criação de um sistema para o design

¹ tiles (ou ladrilhos, em português) são pequenas imagens ou gráficos que são usados para construir o cenário ou mapa do jogo

² <https://www.nintendo.com/pt-pt/Jogos/NES/Super-Mario-Bros-803853.html>

de ambientes de jogos, utilizando geração procedural com inteligência artificial para facilitar a criação de narrativas interativas. O trabalho foca em desenvolver um sistema intuitivo que permite aos usuários criar e experienciar diferentes histórias, automatizando a geração inicial do ambiente, mas permitindo customização posterior. A semelhança com este TCC está na aplicação de LLMs para a geração automática de ambientes de jogo e no objetivo de criar uma ferramenta acessível para criadores. A diferença reside no foco: o trabalho de Auxtero (2023) é mais abrangente, visando jogos de aventura em geral e com grande ênfase na interface de customização manual, enquanto este TCC se concentra nas restrições e particularidades algorítmicas específicas do gênero *roguelike*.

Por fim, Gallotta *et al.* (2024) exploram a geração consistente de conteúdo através de um framework chamado *LLMaker*. A principal contribuição do trabalho é o uso de *function calling*, onde o LLM atua como um tradutor de comandos em linguagem natural para chamadas de funções formais em um sistema de *back-end*. Essa abordagem garante que toda modificação no conteúdo do jogo (neste caso, *layouts* de masmorras) seja válida e adira às restrições do domínio, mitigando o problema de alucinações e inconsistências do LLM. A semelhança com este TCC está no reconhecimento do desafio da consistência na geração de conteúdo e na busca por uma solução que garanta a validade dos artefatos gerados. O domínio de aplicação, focado em *dungeon crawlers*, também é bastante próximo ao gênero *roguelike*. A diferença principal, entretanto, está no paradigma de interação. *LLMaker* é uma ferramenta de codesign iterativo, onde um designer humano interage continuamente com o sistema via chat, enquanto a proposta deste TCC foca em um processo mais automatizado de geração inicial a partir de uma entrada textual.

O Quadro 1 apresenta uma análise comparativa dos trabalhos relacionados mais próximos. A comparação é baseada em critérios relevantes para esta pesquisa, destacando as semelhanças e diferenças entre as abordagens.

Quadro 1 – Comparativo dos Trabalhos Relacionados

Trabalho	Geração de níveis a partir de descrições textuais	Foco em engenharia de prompt	Saída em formato estruturado (e.g., JSON)	Abordagem focada no gênero Roguelike
(NASIR <i>et al.</i> , 2024)	x	x		
(SUDHAKARAN <i>et al.</i> , 2023)	x			
(AUXTERO, 2023)	x		x	
(GALLOTTA <i>et al.</i> , 2024)	x	x		
Trabalho Proposto	x	x	x	x

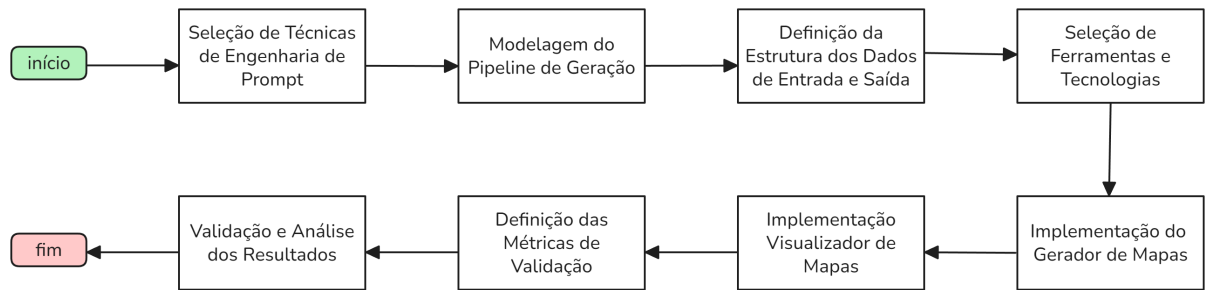
Fonte: Elaborado pelo autor

4 METODOLOGIA

4.1 Visão Geral da Metodologia

A metodologia deste trabalho é concebida como um fluxo de trabalho lógico e progressivo, garantindo que cada etapa se fundamente nas conclusões da anterior. A Figura 7 ilustra a sequência de fases que orientam a pesquisa, desde a concepção teórica até a análise empírica dos resultados. Este percurso metodológico assegura a rastreabilidade e a reprodutibilidade dos procedimentos adotados para desenvolver e validar o sistema de geração de mapas para jogos *roguelike*.

Figura 7 – Fluxograma das etapas metodológicas do trabalho.



Fonte: Elaborado pelo autor

4.2 Seleção de Técnicas de Engenharia de Prompt

A primeira etapa metodológica foca-se na investigação e seleção criteriosa das técnicas de engenharia de prompt. Dado que o projeto não prevê o *fine-tuning* de um Modelo de LLM, abordagem que demanda vastos recursos computacionais e um volume significativo de dados de treinamento, como visto em trabalhos como o de Sudhakaran *et al.* (2023), a engenharia de prompt emerge como a alternativa estratégica e viável. A partir dessa investigação, são selecionadas as técnicas a seguir para compor a espinha dorsal do pipeline de geração.

A primeira técnica adotada é a *Role-Play Prompting* (KONG *et al.*, 2023). Esta abordagem consiste em instruir o LLM a assumir uma persona específica (por exemplo, um "arquiteto de masmorras") para realizar uma determinada tarefa. A pesquisa de Kong *et al.* (2023) demonstra que essa técnica melhora significativamente as capacidades de raciocínio de *zero-shot* dos LLMs, funcionando como um gatilho implícito e eficaz para processos de pensamento mais estruturados, como *Chain-of-Thought*. Ao atribuir papéis especializados em

diferentes estágios do *pipeline*, busca-se especializar a resposta do LLM para cada subtarefa, aumentando a coerência e a qualidade da geração.

A segunda técnica é a *Structured Outputs*, especificamente no formato JSON (SHORTEN *et al.*, 2024). Para que um motor de jogo possa interpretar e instanciar um mapa, a saída do LLM não pode ser apenas texto em linguagem natural; ela precisa ser um artefato de dados bem-definido e passível de análise computacional (*parsable*). O trabalho de Shorten *et al.* (2024) destaca a importância crítica e os desafios de garantir que os LLMs produzam saídas JSON formatadas corretamente, especialmente no contexto de sistemas de IA compostos, onde múltiplos componentes interagem. A adoção de saídas JSON garante a interoperabilidade entre o sistema de geração e o motor de jogo, impondo uma restrição formal que minimiza erros e ambiguidades.

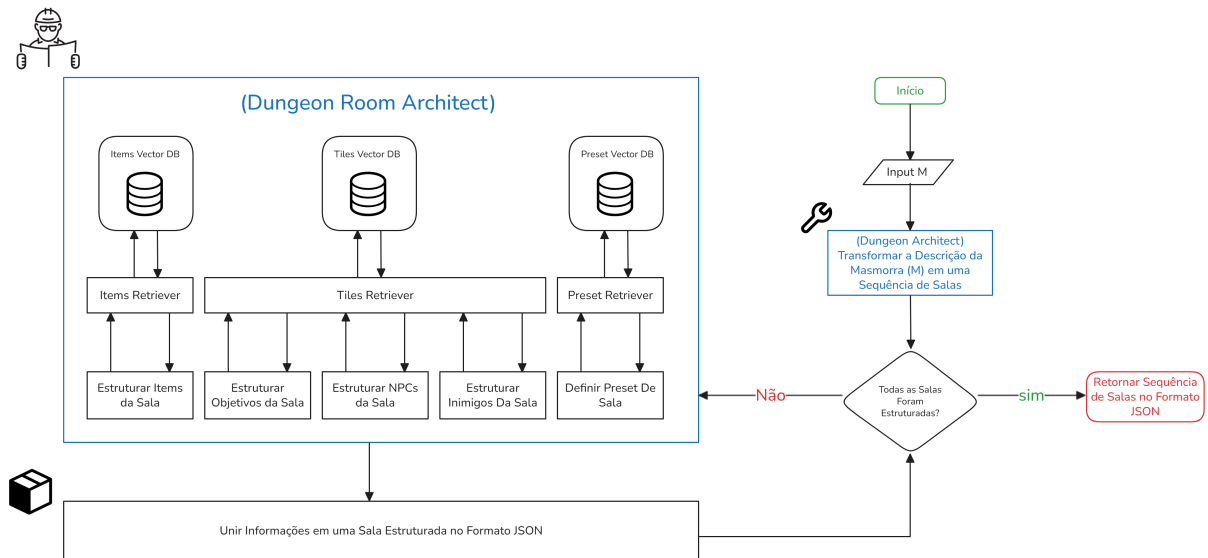
Finalmente, para garantir que o conteúdo gerado (inimigos, itens, objetivos) seja fiel aos ativos existentes no jogo, é selecionada a técnica de RAG (SAHOO *et al.*, 2024). Os LLMs, embora vastos em conhecimento geral, não possuem informações sobre o domínio específico de um jogo. O RAG soluciona essa lacuna ao permitir que o modelo consulte uma ou mais bases de conhecimento externas (neste caso, bancos de dados vetoriais contendo os ativos do jogo) antes de formular uma resposta. Conforme detalhado no levantamento de Sahoo *et al.* (2024), o RAG é uma técnica fundamental para reduzir alucinações e fundamentar as gerações do LLM em dados factuais, garantindo que os elementos do mapa correspondam a entidades realmente existentes no jogo. A combinação sinérgica dessas três técnicas, *Role-Play Prompting* para guiar o comportamento, *Structured Outputs* para garantir a usabilidade e RAG para assegurar a fidelidade factual, constitui a base metodológica para o pipeline proposto.

4.3 Modelagem do Pipeline de Geração

Uma vez definidas as técnicas de engenharia de prompt, a etapa seguinte é a modelagem do pipeline de geração. Com base nas técnicas selecionadas, é modelado um *pipeline* de geração multifásico, projetado para traduzir uma descrição textual de alto nível em uma estrutura de dados de mapa detalhada e pronta para uso. A arquitetura do pipeline, ilustrada no fluxograma da Figura 8, é fundamentada no princípio de especialização de tarefas, utilizando a técnica de *Role-Play Prompting* para atribuir diferentes papéis ao LLM em cada etapa.

O processo inicia-se com uma descrição textual da masmorra. O usuário fornece uma entrada, que é então processada pelo *Dungeon Architect*, o primeiro papel assumido pelo LLM. A responsabilidade deste agente é transformar a narrativa em linguagem natural em uma

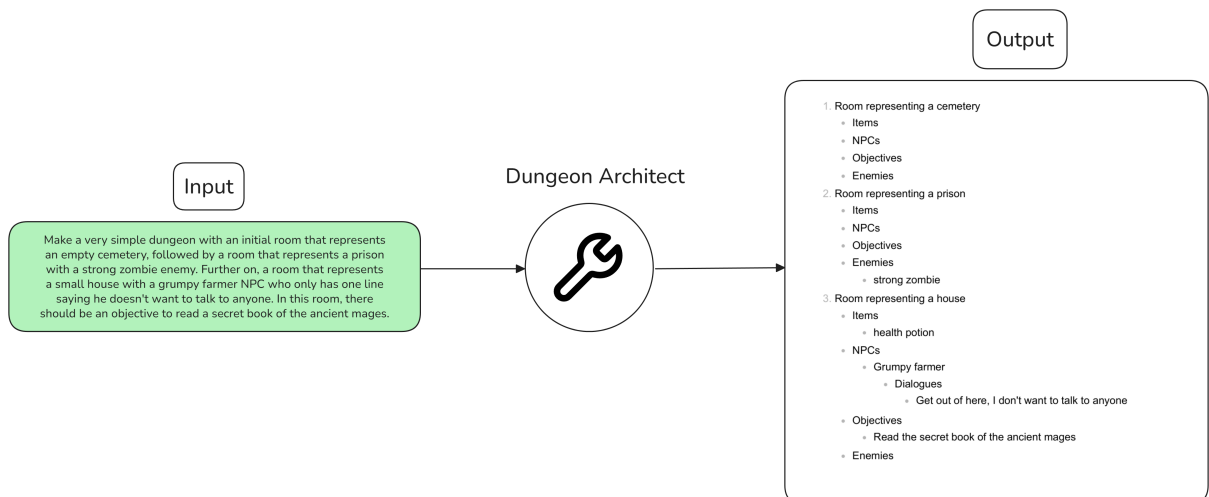
Figura 8 – Pipeline do Gerador de Mapas



Fonte: Elaborado pelo autor

lista sequencial e ordenada de salas. Esta etapa estabelece a macroestrutura do mapa e organiza o fluxo narrativo de forma mais sistemática, servindo como um roteiro para a fase de geração detalhada. A Figura 9 exemplifica esta transformação, mostrando como uma descrição textual é convertida em uma sequência estruturada de salas.

Figura 9 – Exemplo de entrada e saída para o papel *Dungeon Architect*



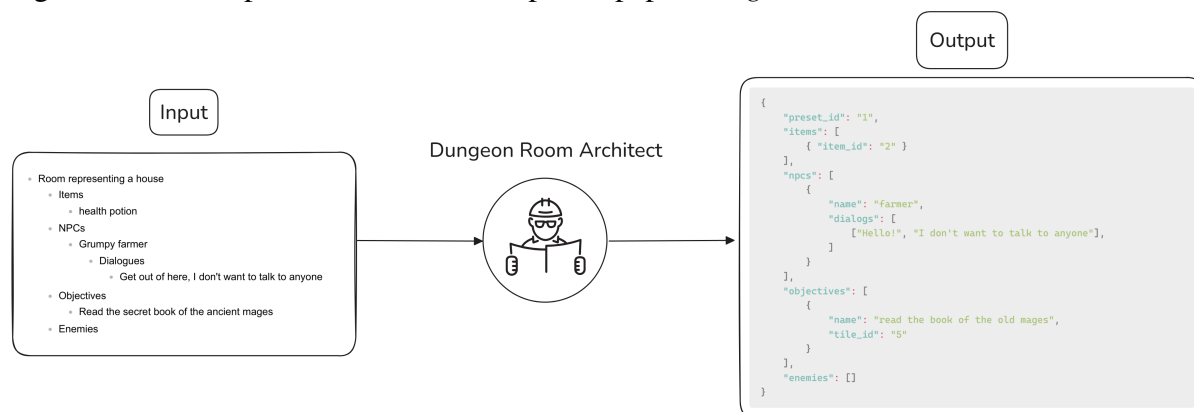
Fonte: Elaborado pelo autor

O núcleo do pipeline opera em um laço que processa cada sala da sequência individualmente. Nesta fase, o LLM assume seu papel mais complexo, o *Dungeon Room Architect*. Este agente é responsável por analisar a descrição de uma única sala e extrair dela todos os elementos relevantes: o tipo de sala (*preset*), os inimigos, os NPCs, os itens e os objetivos. Para garantir que os elementos extraídos correspondam a ativos válidos no jogo, o *Dungeon Room*

Architect emprega a técnica de RAG. Ele consulta três bancos de dados vetoriais distintos (Banco de Presets de Sala, Banco de Tiles e Banco de Itens) para recuperar os identificadores (IDs) únicos associados às descrições textuais.

O resultado da atuação do *Dungeon Room Architect* para cada sala é uma saída estruturada em formato JSON. Este objeto JSON encapsula todas as informações da sala de forma organizada e inequívoca, utilizando os IDs recuperados pelo RAG para referenciar os ativos do jogo, como ilustrado na Figura 10. Ao final do laço, o pipeline entrega uma sequência de objetos JSON, cada um representando uma sala do calabouço. Esta estrutura de dados final, por ser estritamente definida e computacionalmente interpretável, está pronta para ser consumida por um motor de jogo para a renderização e instanciação do nível jogável.

Figura 10 – Exemplo de entrada e saída para o papel *Dungeon Room Architect*



Fonte: Elaborado pelo autor

4.4 Definição das Estruturas de Dados de Entrada e Saída

Uma etapa fundamental para a interação bem-sucedida com um LLM é a definição precisa das estruturas de dados que governam tanto a entrada do usuário quanto a saída gerada pelo modelo. Esta fase concentra-se na especificação formal dos formatos utilizados na comunicação. Enquanto a entrada principal consiste em uma descrição textual em linguagem natural, a saída que representa o mapa do jogo é formalizada em JSON.

A escolha pelo JSON para a saída estruturada deve-se à sua legibilidade, amplo suporte em diversas linguagens de programação e facilidade de manipulação por LLMs. Foram definidos esquemas para todos os elementos constituintes do mapa, como salas (com suas dimensões e propriedades), inimigos (com seus atributos), itens, NPCs e objetivos. A formalização dessas estruturas de saída é fundamental, pois não apenas formatam o resultado final, mas

também são essenciais para técnicas de *prompt* como o *Few-shot learning*, onde exemplos no formato desejado são fornecidos ao modelo. Isso força o LLM a aderir ao esquema predefinido, minimizando erros e inconsistências.

4.5 Seleção de Ferramentas e Tecnologias

Para a materialização do sistema, é necessário selecionar um conjunto de ferramentas e tecnologias adequadas. Esta seleção será guiada por critérios de acessibilidade, maturidade e alinhamento com os objetivos do projeto. As escolhas tecnológicas abrangerão: os modelos de LLMs a ser utilizado, cuja decisão considerará a disponibilidade de *Application Programming Interface* (API), custo e performance; o motor de jogo para a visualização e interpretação dos mapas gerados; as linguagens de programação e as bibliotecas de suporte para a comunicação com o LLM e para a manipulação dos dados; e os recursos visuais, como os conjuntos de *tiles* que comporão graficamente os elementos do mapa (paredes, pisos, personagens, itens).

4.6 Implementação do Gerador de Mapas

Com todos os pré-requisitos definidos (técnicas de *prompt*, arquitetura do pipeline, estruturas de dados e tecnologias), a etapa seguinte é a implementação do gerador de mapas. Esta fase de desenvolvimento de software consiste em traduzir o projeto conceitual em código, construindo um sistema capaz de executar o pipeline de geração. O sistema integrará a comunicação com a API do LLM, aplicará as estratégias de engenharia de *prompt* e produzirá um arquivo de mapa no formato JSON definido, que poderá ser subsequentemente processado pela etapa de visualização. Esta fase, portanto, concentra-se na lógica de interação com o modelo de linguagem e na produção dos dados brutos que descrevem o mapa.

4.7 Implementação do Visualizador de Mapas

Após a geração da representação estruturada do mapa em formato JSON, a etapa seguinte consiste na implementação de um visualizador. Este componente é fundamental para traduzir os dados abstratos em um artefato tangível e inspecionável, servindo como a principal ferramenta para a validação qualitativa dos resultados. O propósito do visualizador não é criar um jogo completo, mas sim renderizar o mapa gerado de forma fiel, permitindo uma análise clara de sua estrutura, composição e conformidade com a descrição textual original.

A implementação será realizada utilizando o motor de jogo e os recursos visuais (*tilesets*) selecionados na etapa de Seleção de Ferramentas e Tecnologias. O visualizador será responsável por ler e interpretar o arquivo JSON gerado pelo protótipo. Em seguida, percorrerá a estrutura de dados, instanciando os elementos do mapa. Este processo materializa o mapa, tornando possível verificar visualmente a conectividade entre as salas, a distribuição dos elementos e a coerência geral do layout, aspectos essenciais para a fase de avaliação.

4.8 Definição das Métricas de Validação

Para validar a eficácia e a qualidade do sistema gerador, é indispensável estabelecer um conjunto de métricas de validação claras e objetivas. Tendo a estrutura do projeto e os esquemas de dados definidos, esta fase se dedica a formalizar os critérios que serão utilizados para analisar os mapas gerados. As métricas serão divididas em categorias, como: (i) **Validade Estrutural**, que verifica se a saída do LLM é um JSON sintaticamente correto e se adere ao esquema definido; e (ii) **Fidelidade Semântica**, que valida o grau de alinhamento entre os elementos presentes no mapa gerado e a descrição textual original.

Como referência o trabalho de Nasir *et al.* (2024) emprega uma combinação de avaliações, tanto baseadas em LLM quanto em métodos convencionais de PCG, para medir a fidelidade e a qualidade dos mundos gerados. Por exemplo, eles utilizam uma métrica de Coerência, na qual o próprio LLM avalia o quão bem o mundo gerado se alinha com a história original, considerando o ambiente, os personagens e os objetivos. Adicionalmente, o trabalho mede a precisão da colocação de personagens e de itens importantes, que calcula a proporção de elementos cruciais da história que foram efetivamente inseridos no mapa.

4.9 Validação e Análise dos Resultados

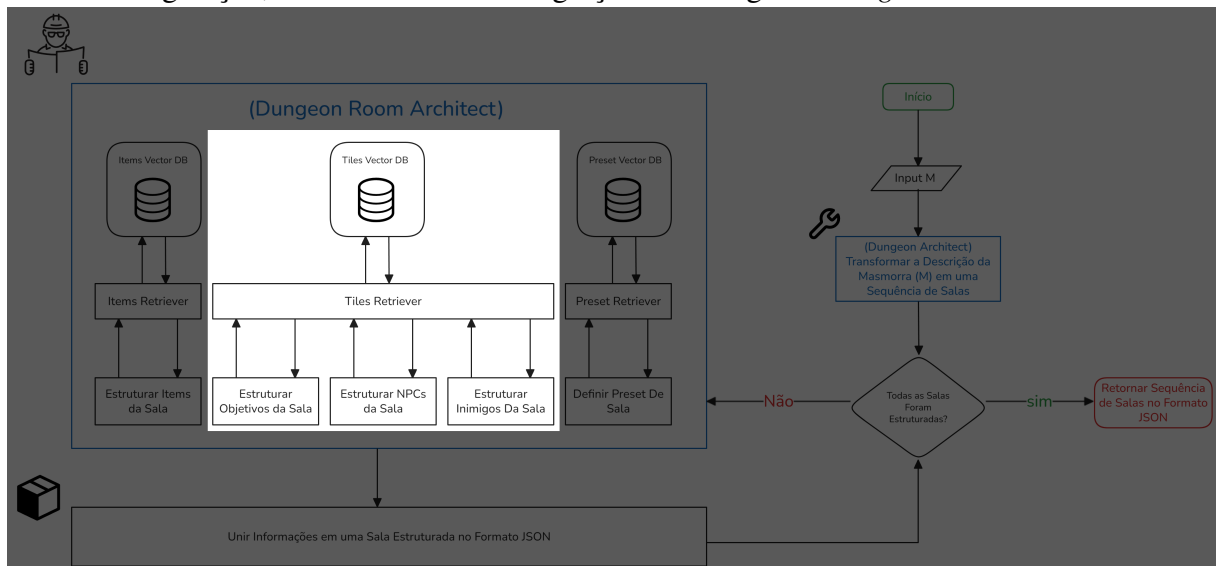
A etapa final da metodologia consiste na execução de uma série de testes para validar o sistema com base nas métricas previamente definidas. Um conjunto diversificado de descrições textuais será fornecido como entrada ao sistema, e os mapas resultantes serão sistematicamente analisados. Esta análise permitirá aferir a qualidade das gerações, identificar os pontos fortes e as limitações da abordagem proposta, e compreender o impacto das escolhas de engenharia de prompt nos resultados. A conclusão desta fase fornecerá os subsídios empíricos necessários para discutir as contribuições do trabalho e sugerir direções para pesquisas futuras.

5 RESULTADOS PRELIMINARES

5.1 Implementação da Primeira Versão do Banco de Tiles

Como parte dos resultados preliminares para a construção do pipeline de geração de mapas, um componente essencial do agente *Dungeon Room Architect* foi desenvolvido: o banco de dados de *tiles*. Este componente é fundamental para a técnica de RAG, permitindo que o sistema traduza descrições semânticas de partes de uma sala em elementos visuais concretos e existentes no acervo do jogo. A sua função é garantir que, ao receber uma descrição como "um piso de pedra rachado com musgo", o sistema possa recuperar o *tile* mais apropriado para representar visualmente essa intenção. A Figura 11 destaca a localização e o papel deste banco de dados vetorial dentro do fluxograma geral de geração.

Figura 11 – Destaque do componente do Banco de Tiles Vetorial no fluxograma do pipeline de geração, evidenciando sua integração com o agente *Dungeon Room Architect*.



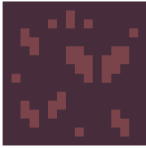
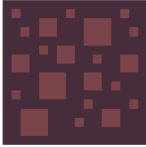
Fonte: Elaborado pelo autor

A base para este sistema foi construída a partir de um arquivo no formato *Comma Separated Values* (CSV) contendo um conjunto de 1072 *tiles*¹ distintos. Cada linha nesse arquivo representa um único *tile* e é estruturada com as seguintes informações: a representação da imagem no formato PNG codificada em base64; as coordenadas X e Y relativas à sua posição em um *tileset* mestre; e, fundamentalmente, um conjunto de cinco descrições textuais curtas que capturam diferentes aspectos e sinônimos do elemento visual. Esta multiplicidade de descrições visa enriquecer a representação semântica de cada *tile*, aumentando a robustez do sistema de

¹ Conjunto de *tiles* utilizados: <https://kenney.nl/assets/1-bit-pack>

busca. A Figura 12 exibe um exemplo ilustrativo da estrutura de dados para algumas instâncias de *tiles*.

Figura 12 – Exemplo de entradas no arquivo CSV de descrições de *tiles*.

#	Image	Descriptions	Coords
0		• a dark purple gem • a patch of shadow • a bruised skin texture • a magic portal • a UI element background	(0,0)
1		• scattered bloodstains • a patch of red mushrooms • a cluster of rubies • a damaged leather patch • a small fire's embers	(1,0)
2		• a field of small stones • a rough dirt path • a bloodstained ground • a texture for a dark cave floor • a damaged stone wall	(2,0)

Fonte: Elaborado pelo autor

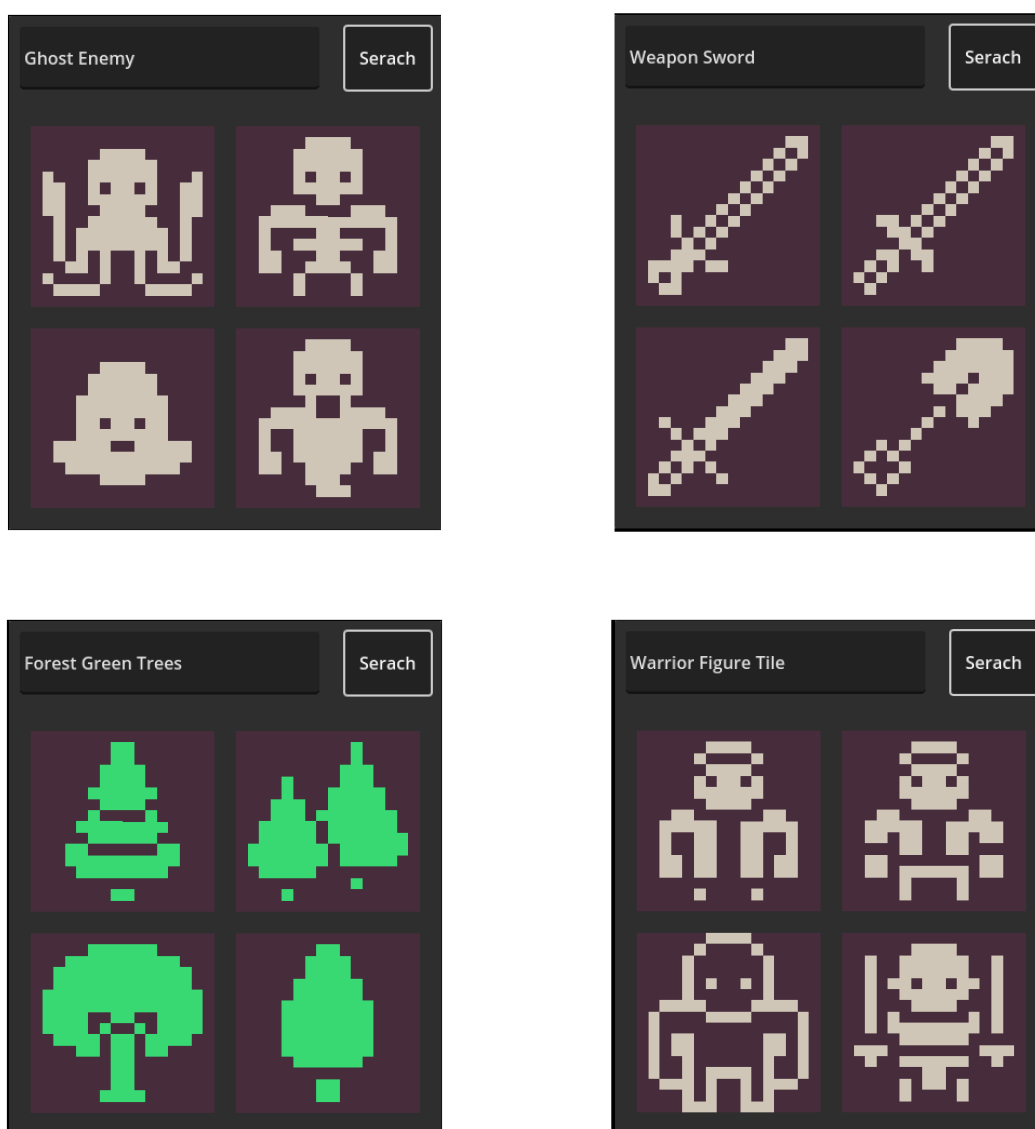
Para viabilizar a busca textual de forma eficiente, as instâncias deste CSV foram processadas e compactadas em um *Vector Database*. Neste processo, as cinco descrições de cada *tile* são concatenadas e transformadas em um único vetor de *embeddings* por meio de um modelo de linguagem pré-treinado. O *embedding* resultante é uma representação numérica de alta dimensão que encapsula o significado semântico das descrições. Desta forma, o *Vector Database* armazena um vetor para cada um dos 1072 *tiles*, permitindo comparações matemáticas de similaridade.

O mecanismo de consulta opera de maneira análoga. Quando uma consulta textual é enviada ao sistema (por exemplo, "chão de terra com ossos") ela é primeiramente convertida no mesmo formato de *embedding*. Subsequentemente, o *Vector Database* realiza uma análise de similaridade de cosseno entre o *embedding* da consulta e todos os *embeddings* de *tiles* armazenados. Como resultado, o sistema retorna os *tiles* cujos vetores são mais próximos ao vetor da consulta no espaço vetorial, ou seja, aqueles cujas descrições são semanticamente mais similares à consulta realizada.

Para demonstrar e validar o funcionamento deste componente, foi implementada uma API que expõe a funcionalidade de busca e um programa com uma GUI. Esta ferramenta permite que um usuário insira uma consulta textual em um campo de entrada e receba, como resposta visual, os quatro *tiles* mais relevantes encontrados no banco de dados. A Figura 13 ilustra a

interface em funcionamento, apresentando os resultados para quatro exemplos de consultas distintas. Esta implementação serve como uma prova de conceito funcional da capacidade do sistema de associar linguagem natural a ativos de jogo específicos, constituindo um pilar fundamental para a abordagem *"text-to-map"* proposta neste trabalho.

Figura 13 – Interface gráfica da ferramenta de busca de tiles, exibindo os resultados para quatro consultas distintas. Para cada consulta, são retornados os quatro tiles mais semanticamente similares.



Fonte: Elaborado pelo autor

6 PLANO DE TRABALHO

Para consolidar o planejamento e a execução desse trabalho, apresenta-se o cronograma proposto para o desenvolvimento do projeto. O Quadro 2 ilustra a distribuição temporal das principais atividades, organizadas em uma sequência lógica que se estende de junho a dezembro. Este planejamento visa garantir que todas as fases, desde a modelagem do pipeline e implementação dos artefatos de software até a avaliação dos resultados, sejam cumpridas de forma estruturada e dentro do prazo previsto.

Quadro 2 – Cronograma de atividades

 Finalizada  Pendente

Atividades	JUN	JUL	AGO	SET	OUT	NOV	DEZ
Escrita do TCC							
Seleção de Técnicas de Engenharia de Prompt							
Modelagem do Pipeline de Geração							
Definição da Estrutura dos Dados de Saída							
Seleção de Ferramentas e Tecnologias							
Implementação do Protótipo Gerador							
Implementação Visualizador de Mapas							
Definição das Métricas de Avaliação							
Avaliação e Análise dos Resultados							

Fonte: Elaborado pelo autor

REFERÊNCIAS

- AUXTERO, A. L. S. **Game Environment Design Creator Using Artificial Intelligence Procedural Generation**. Dissertação (Mestrado) — Universidade NOVA de Lisboa (Portugal), 2023.
- BOTTA, C. L. B. Oh no, i'm dead!... again. 2011.
- BROWN, P. F.; PIETRA, V. J. D.; DESOUSA, P. V.; LAI, J. C.; MERCER, R. L. Class-based n-gram models of natural language. **Computational linguistics**, v. 18, n. 4, p. 467–480, 1992.
- DEVLIN, J.; CHANG, M.-W.; LEE, K.; TOUTANOVA, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In: **Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)**. [S.l.: s.n.], 2019. p. 4171–4186.
- EHRIG, H.; PFENDER, M.; SCHNEIDER, H. J. Graph-grammars: An algebraic approach. In: IEEE. **14th Annual symposium on switching and automata theory (swat 1973)**. [S.l.], 1973. p. 167–180.
- FREIKNECHT, J. **Procedural Content Generation for Games**. Tese (Inauguraldissertation zur Erlangung des akademischen Grades eines Doktors der Naturwissenschaften) — Universität Mannheim, Mannheim, 2020. Data da defesa oral: 12 de Fevereiro de 2021.
- GALLOTTA, R.; LIAPIS, A.; YANNAKAKIS, G. Consistent game content creation via function calling for large language models. In: IEEE. **2024 IEEE Conference on Games (CoG)**. [S.l.], 2024. p. 1–4.
- HADI, M. U.; QURESHI, R.; SHAH, A.; IRFAN, M.; ZAFAR, A.; SHAIKH, M. B.; AKHTAR, N.; WU, J.; MIRJALILI, S. *et al.* A survey on large language models: Applications, challenges, limitations, and practical usage. **Authorea Preprints**, Authorea, 2023.
- HARRIS, J. **Exploring roguelike games**. [S.l.]: CRC Press, 2020.
- HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural computation**, MIT press, v. 9, n. 8, p. 1735–1780, 1997.
- KOESNAEDI, A.; ISTIONO, W. Implementation drunkard's walk algorithm to generate random level in roguelike games. **International Journal of Multidisciplinary Research and Publications**, v. 5, n. 2, p. 97–103, 2022.
- KONG, A.; ZHAO, S.; CHEN, H.; LI, Q.; QIN, Y.; SUN, R.; ZHOU, X.; WANG, E.; DONG, X. Better zero-shot reasoning with role-play prompting. **arXiv preprint arXiv:2308.07702**, 2023.
- KOPPEN, L. **Integrating a Human Feedback Loop in PCG for Level Design Using LLMs**. Dissertação (B.S. thesis) — University of Twente, 2024.
- LAURINDO, E. K.; OTI, K. H.; BROCATTO, M. d. A.; BELCHIOR, V. Relatório técnico de desenvolvimento do jogo digital thorn ascent. 004, 2024.
- LAZARIDIS, L.; FRAGULIS, G. F. Creating a newer and improved procedural content generation (pcg) algorithm with minimal human intervention for computer gaming development. **Computers**, MDPI, v. 13, n. 11, p. 304, 2024.

- LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, Ieee, v. 86, n. 11, p. 2278–2324, 2002.
- LEVIN, D. A.; PERES, Y. **Markov chains and mixing times**. [S.l.]: American Mathematical Soc., 2017. v. 107.
- LIN, T.; WANG, Y.; LIU, X.; QIU, X. A survey of transformers. **AI open**, Elsevier, v. 3, p. 111–132, 2022.
- LINDENMAYER, A. Mathematical models for cellular interactions in development i. filaments with one-sided inputs. **Journal of theoretical biology**, Elsevier, v. 18, n. 3, p. 280–299, 1968.
- LIU, Y.; HE, H.; HAN, T.; ZHANG, X.; LIU, M.; TIAN, J.; ZHANG, Y.; WANG, J.; GAO, X.; ZHONG, T. *et al.* Understanding llms: A comprehensive overview from training to inference. **Neurocomputing**, Elsevier, p. 129190, 2024.
- LOH, W.-Y. Classification and regression trees. **Wiley interdisciplinary reviews: data mining and knowledge discovery**, Wiley Online Library, v. 1, n. 1, p. 14–23, 2011.
- MALEKI, M. F.; ZHAO, R. Procedural content generation in games: A survey with insights on emerging llm integration. In: **Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment**. [S.l.: s.n.], 2024. v. 20, n. 1, p. 167–178.
- MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **The bulletin of mathematical biophysics**, Springer, v. 5, n. 4, p. 115–133, 1943.
- NASIR, M. U.; JAMES, S.; TOGELIUS, J. Word2world: Generating stories and worlds through large language models. **arXiv preprint arXiv:2405.06686**, 2024.
- NAVEED, H.; KHAN, A. U.; QIU, S.; SAQIB, M.; ANWAR, S.; USMAN, M.; AKHTAR, N.; BARNES, N.; MIAN, A. A comprehensive overview of large language models. **arXiv preprint arXiv:2307.06435**, 2023.
- PEARSON, K. The problem of the random walk. **Nature**, Nature Publishing Group UK London, v. 72, n. 1867, p. 342–342, 1905.
- PERLIN, K. An image synthesizer. **ACM Siggraph Computer Graphics**, ACM New York, NY, USA, v. 19, n. 3, p. 287–296, 1985.
- PERLIN, K. Noise hardware. In: OLANO, M. (Ed.). **Real-Time Shading SIGGRAPH Course Notes**. [S.l.]: ACM SIGGRAPH, 2001. Course Notes.
- RADFORD, a.; NARASIMHAN, k.; SALIMANS, t.; SUTSKEVER, i. *et al.* improving language understanding by generative pre-training. san francisco, ca, usa, 2018.
- ROLIM, F. M. Llms e ia generativa em jogos. Universidade Federal do Rio de Janeiro, 2023.
- SAHOO, P.; SINGH, A. K.; SAHA, S.; JAIN, V.; MONDAL, S.; CHADHA, A. A systematic survey of prompt engineering in large language models: Techniques and applications. **arXiv preprint arXiv:2402.07927**, 2024.
- SEPÄNMAA, T. Procedural level generation in 2d roguelite games. 2024.
- SHORT, T.; ADAMS, T. **Procedural generation in game design**. [S.l.]: CRC Press, 2017.

SHORTEN, C.; PIERSE, C.; SMITH, T. B.; CARDENAS, E.; SHARMA, A.; TRENGROVE, J.; LUIJT, B. van. Structuredrag: Json response formatting with large language models. **arXiv preprint arXiv:2408.11061**, 2024.

SMITH, G.; GAN, E.; OTHENIN-GIRARD, A.; WHITEHEAD, J. Pcg-based game design: enabling new play experiences through procedural content generation. In: **Proceedings of the 2nd International Workshop on Procedural Content Generation in Games**. [S.l.: s.n.], 2011. p. 1–4.

SOUZA, R.; JUNIOR, B.; FOSS, L.; CAVALHEIRO, G. G.; CAVALHEIRO, S. Geração procedural de mapas dungeon crawl baseada em gramática de grafos para uso em jogos roguelike. In: SBC. **Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD)**. [S.l.], 2019. p. 193–203.

STINY, G.; GIPS, J. Shape grammars and the generative specification of painting and sculpture. In: **IFIP congress (2)**. [S.l.: s.n.], 1971. v. 2, n. 3, p. 125–135.

SUDHAKARAN, S.; GONZÁLEZ-DUQUE, M.; FREIBERGER, M.; GLANOIS, C.; NAJARRO, E.; RISI, S. MarioGPT: Open-ended text2level generation through large language models. **Advances in Neural Information Processing Systems**, v. 36, p. 54213–54227, 2023.

TOGELIUS, J.; KASTBJERG, E.; SCHEDL, D.; YANNAKAKIS, G. N. What is procedural content generation? mario on the borderline. In: **Proceedings of the 2nd international workshop on procedural content generation in games**. [S.l.: s.n.], 2011. p. 1–6.

TOGELIUS, J.; YANNAKAKIS, G. N.; STANLEY, K. O.; BROWNE, C. Search-based procedural content generation: A taxonomy and survey. **IEEE Transactions on Computational Intelligence and AI in Games**, IEEE, v. 3, n. 3, p. 172–186, 2011.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. **Advances in neural information processing systems**, v. 30, 2017.

WERNECK, M.; CLUA, E. W. Generating procedural dungeons using machine learning methods. In: IEEE. **2020 19th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)**. [S.l.], 2020. p. 90–96.

WIJMAN, T. Newzoo's year in review: the 2023 global games market in numbers. 2023. Disponível em: <<https://newzoo.com/resources/blog/video-games-in-2023-the-year-in-numbers>>.